Taller de Algoritmos y Estructura de Datos II · Universidad Siglo 21

RedSens

Sistema de gestión y mantenimiento de la red provincial de estaciones de medición de DATA Investigaciones

Informe técnico · Trabajo Práctico Integral

Grupo	2
Parámetros	Semilla 2026 · tabla hash inicial 13 · base Villa María · M de Josephus 2
Lenguaje	Java 17 (aplicación de consola, sin librerías externas)
Integrantes	Valentino · Bruno A · Nacho · Lautaro · Bruno B · Aitana

1. Descripción de la solución

RedSens es una aplicación de consola organizada en un paquete por módulo. Los seis módulos comparten el modelo `Estacion` y funcionan como un único sistema desde el menú principal (`redsens.Main`):

Paquete	Responsabilidad	Clases principales
modelo	Datos de una estación y parámetros del grupo	Estacion, Parametros
m1	Búsqueda, alta y baja por código en O(1) promedio	TablaHash, TablaHashEncadenada, ExperimentoHash
m2	Índice ordenado por número: orden, rangos, críticas	ArbolBinarioBusqueda, ABB, AVL, ExperimentoArboles
m3	Red vial y recorridos de las cuadrillas	Grafo, GrafoMatriz, GrafoLista, AlgoritmosGrafo, PlanVisitas
m4	Compresión sin pérdida del archivo de mediciones	Huffman, ArchivoHuffman
m5	Calendario de guardias justo y verificable	JosephusSimple, JosephusArbol, ArbolRango, Guardias
m6	Datos simulados reproducibles y medición de rendimiento	GeneradorDatos, PruebasAleatorios, Cronometro, InformeRendimiento

Flujo integrado. «Cargar la red» genera N estaciones con la semilla del grupo (M6) y las inserta a la vez en la tabla hash (M1) y en el AVL (M2). La tabla hash atiende las búsquedas por código; el AVL, los listados en orden, los rangos y el reporte de críticas. Ese reporte se agrupa por localidad y alimenta el plan de visitas del M3, que usa la tabla de distancias de Floyd. El M4 genera el archivo de mediciones a partir de las estaciones cargadas, y el M6 reúne los experimentos de M1, M2 y M5 en un informe de rendimiento que el programa guarda en un archivo.

Por qué dos estructuras para los mismos datos. La tabla hash es la mejor para «¿dónde está CBA-0347?» (O(1) promedio), pero desordena las claves a propósito: no sirve para listar en orden ni por rango. El AVL resuelve eso en O(log n + k) a cambio de búsquedas O(log n). Mantener ambas cuesta el doble de inserciones, pero cada consulta usa la estructura que le conviene.

Restricciones. M1, M2 y M3 no usan `HashMap`, `TreeMap` ni librerías de grafos: la tabla, los árboles, las listas de adyacencia y la cola del BFS están implementadas con arreglos y nodos propios.

2. M1 · Registro de estaciones (tabla hash)

Diseño

- **Direccionamiento abierto** con un arreglo de celdas. Cada celda guarda la estación y un indicador `activa`. La estrategia (lineal o cuadrática) se elige al crear la tabla.
- **Exploración lineal:** h, h+1, h+2, … **Cuadrática:** h, h+1, h+4, h+9, …, calculada de forma incremental ($i^2 - (i-1)^2 = 2i - 1$) para evitar multiplicaciones y desbordes.
- **Eliminación perezosa:** la celda se marca como borrada y no se vacía. Si se vaciara, una búsqueda de una clave que se insertó después, y que tuvo que saltar esa celda, cortaría la secuencia de sondeo y la daría por inexistente. Las celdas borradas siguen contando como ocupadas para el factor de carga y se descartan en el rehashing.
- **Rehashing:** cuando las celdas ocupadas superan `umbral × tamaño`, la tabla pasa al primer primo mayor o igual que el doble y se reinsertan solo las activas. Con tamaño inicial 13: 13 → 29 → 59 → … → 33.703.

- **Contadores:** colisiones acumuladas en las inserciones (cada celda ocupada por otra clave que se encuentra en el camino) y sondeos de la última operación (celdas examinadas).

Función hash

```
h = 0
para cada carácter c del código:  h = 31·h + c      // Horner: CBA-1234 ≠ CBA-4321
h ^= h >>> 16;  h *= 0x45d9f3b;  h ^= h >>> 16       // mezcla de bits
posición = floorMod(h, tamaño)                       // tamaño primo; floorMod evita negativos
```

El polinomio de Horner usa todos los caracteres y los pondera por potencias distintas de 31, así que importa el orden de los dígitos. La mezcla se agregó después de medir (ver tabla): con Horner solo, CBA-0347 y CBA-0348 dan valores que difieren en 1 y caen en celdas vecinas, de modo que códigos correlativos forman bloques contiguos. Eso es agrupamiento primario fabricado por la propia función, y con exploración lineal los sondeos se disparaban hasta 3 veces por encima de la teoría. Multiplicar por una constante impar grande y combinar bits altos y bajos desparrama esos valores. El tamaño primo hace que el módulo use todos los bits del valor y es condición para la garantía de la cuadrática.

Umbrales elegidos

- **Lineal: 0,5.** Una búsqueda fallida cuesta $\frac{1}{2}(1 + 1/(1-\lambda)^2)$ sondeos: 2,5 con $\lambda = 0{,}5$, pero 8,5 con $\lambda = 0{,}75$ y 50,5 con $\lambda = 0{,}9$. Por el agrupamiento primario el costo crece muy rápido pasado 0,5.
- **Cuadrática: 0,5.** Con tamaño primo y $\lambda \leq 0{,}5$ está demostrado que siempre se encuentra una celda libre. Por encima, la inserción puede fallar aun con lugar (por seguridad, si recorre la tabla sin éxito se fuerza un rehashing).
- **Encadenamiento: 1,0.** Las listas no se llenan; con $\lambda = 1$ cada lista tiene en promedio un elemento.

Experimento obligatorio

10.000 estaciones simuladas, tamaño inicial 13. Las búsquedas exitosas buscan las 10.000 claves; como con 10.000 estaciones se usan todos los códigos CBA-0000 a CBA-9999, las fallidas buscan 10.000 códigos `XYZ-NNNN`, que no existen.

Configuración	Tamaño final	λ	Rehash	Colisiones	Sondeos éxito	Sondeos fracaso	Teórico éxito / fracaso
Lineal, umbral 0,5	33.703	0,297	11	7.707	1,205	1,494	1,211 / 1,511
Cuadrática, umbral 0,5	33.703	0,297	11	6.546	1,197	1,451	1,186 / 1,422
Encadenamiento, umbral 1,0 (opcional)	16.843	0,594	10	5.052	1,302	0,592	1,297 / 0,594
Lineal, umbral 0,9	16.843	0,594	10	68.415	1,789	3,728	1,731 / 3,529
Cuadrática, umbral 0,9	16.843	0,594	10	27.977	1,600	2,681	1,517 / 2,461
Lineal 0,5, Horner *sin mezcla*	33.703	0,297	11	18.585	3,112	2,339	1,211 / 1,511
Lineal 0,9, Horner *sin mezcla*	16.843	0,594	10	255.299	2,202	11,672	1,731 / 3,529
Lineal 0,5, suma de caracteres	33.703	0,297	11	49.815.000	4.982,5	9.914,0	—

Teórico: lineal $\frac{1}{2}(1+1/(1-\lambda))$ y $\frac{1}{2}(1+1/(1-\lambda)^2)$; cuadrática $\approx -\ln(1-\lambda)/\lambda$ y $1/(1-\lambda)$; encadenamiento $1+\lambda/2$ y λ (nodos comparados).

Análisis

- **Agrupamiento primario.** Con el mismo λ = 0,594, la lineal tiene 2,4 veces más colisiones que la cuadrática (68.415 contra 27.977) y casi un 40 % más de sondeos en las búsquedas fallidas (3,73 contra 2,68). En la lineal, toda clave que cae en cualquier celda de un bloque ocupado termina al final del bloque y lo agranda: los bloques crecen solos. La cuadrática salta cada vez más lejos y no se pega al bloque. Con λ = 0,30 la diferencia casi no se nota, que es justamente lo que busca el umbral 0,5.
- **Medido contra teoría.** Con la función final, los valores medidos coinciden con las fórmulas (por ejemplo 1,79 contra 1,73): la función reparte como se supone en el análisis.
- **La función importa tanto como la estrategia.** Sin la mezcla, la misma lineal con λ = 0,59 hace 11,7 sondeos por búsqueda fallida en lugar de 3,7, y casi el triple de colisiones (255.299). La suma de caracteres solo puede dar 37 valores distintos para CBA-NNNN (la suma de 4 dígitos va de 0 a 36): 10.000 claves se apilan en 37 celdas y la tabla degenera en una búsqueda lineal (casi 5.000 sondeos).
- **Rehashings.** 11 con umbral 0,5 y 10 con 0,9: duplicar hace que el costo total de reinsertar sea O(n) amortizado.

3. M2 · Índice ordenado (ABB y AVL)

Diseño

Una clase abstracta `ArbolBinarioBusqueda` contiene lo común: búsqueda, recorridos inorden, preorden y postorden, altura, cantidad de nodos y de hojas, consulta por rango y reporte de críticas. `ABB` y `AVL` definen inserción y eliminación. La clave es el número de estación.

- **Eliminación en los tres casos:** hoja (se quita), un hijo (el hijo ocupa su lugar) y dos hijos (se reemplaza por el sucesor inorden, el mínimo del subárbol derecho, y se elimina ese mínimo, que tiene a lo sumo un hijo).
- **AVL:** cada nodo guarda su altura. Al volver de la recursión de inserción o eliminación se recalcula el factor de equilibrio `fe = altura(izq) − altura(der)`. Si vale 2 y el hijo izquierdo tiene fe ≥ 0, rotación simple a derecha (caso izquierda-izquierda); si el hijo tiene fe < 0, rotación doble izquierda-derecha. Simétrico para −2.
- **Métodos recursivos y casos base:** `altura` → árbol vacío vale −1; `cantidadNodos` → árbol vacío vale 0; `cantidadHojas` → árbol vacío vale 0 y nodo sin hijos vale 1. Están marcados en el código.
- **Rango [A, B]:** inorden podado: no baja a la izquierda si la clave ya es ≤ A ni a la derecha si ya es ≥ B. Cuesta O(h + k), con k resultados.
- **Críticas:** la batería no es la clave, así que no se puede podar: se recorre todo en inorden (O(n)) y salen ordenadas por número. Con la red de 10.000 estaciones hay 1.950 críticas (≈ 20 %, lo esperable con batería uniforme).

> **Recursión con el ABB degenerado.** Con claves ordenadas el ABB es una lista de 10.000 niveles. La inserción y la búsqueda del ABB son iterativas, y el programa corre en un hilo con 512 MB de pila para que los métodos recursivos (altura, recorridos, eliminar) no produzcan `StackOverflowError`. Es el mismo problema que el enunciado señala para Python (`RecursionError`), resuelto de las dos maneras.

Experimento obligatorio

Orden	N	Altura ABB	Altura AVL	log2 N	Búsqueda prom. ABB	Búsqueda prom. AVL	Inserción total ABB	Inserción total AVL
Ascendente	1.000	999	9	9,97	556 ns	55 ns	5,3 ms	0,4 ms
Ascendente	5.000	4.999	12	12,29	2.851 ns	81 ns	19,6 ms	0,7 ms
Ascendente	10.000	9.999	13	13,29	5.548 ns	79 ns	56,7 ms	1,7 ms
Aleatorio	1.000	19	11	9,97	11 ns	11 ns	0,06 ms	0,21 ms
Aleatorio	5.000	25	13	12,29	45 ns	49 ns	0,36 ms	0,60 ms
Aleatorio	10.000	28	15	13,29	61 ns	62 ns	1,02 ms	2,00 ms

Altura en aristas (una hoja sola tiene altura 0). Búsqueda promedio: se buscan todas las claves y se divide el mejor de 3 tiempos por N.

Análisis

- **Orden ascendente (el caso real del censo):** el ABB degenera en una lista; la altura es N − 1 y buscar es O(n). Con 10.000 estaciones cada búsqueda tarda 70 veces más que en el AVL, y construir el ABB es $O(n^2)$: 56,7 ms contra 1,7 ms.
- **El AVL con claves ordenadas queda casi perfecto** (altura 13 = ⌊log2 10.000⌋): cada inserción al final dispara una rotación simple que mantiene el árbol lleno. Siempre se cumple la cota altura < 1,44·log2(N+2).
- **Orden aleatorio:** el ABB queda razonable (altura 28, unas 2,1·log2 N, como predice el análisis del caso promedio, ≈ 1,39·log2 N de profundidad media) y busca tan rápido como el AVL. Insertar en el AVL cuesta más (recalcular alturas y rotar), pero el AVL garantiza O(log n) sin depender del orden de llegada, que es lo que no controlamos.

4. M3 · Red vial y planificación de recorridos

Representación

El grafo se carga desde `datos/rutas.txt` (líneas `localidad;localidad;km`) y se construyen las dos representaciones. Los algoritmos usan la interfaz `Grafo`, así que corren igual sobre cualquiera (desde el menú se cambia cuál usan). Las listas de adyacencia se mantienen ordenadas por índice de destino para que BFS y DFS visiten los vecinos en el mismo orden que con la matriz y den resultados idénticos.

	Matriz de adyacencia	Lista de adyacencia
Memoria	V^2 = 144 celdas (36 con tramo, 75 % vacío)	V + 2E = 12 + 36 nodos
¿Hay tramo u–v?	O(1)	O(grado)
Recorrer vecinos	O(V) por vértice → BFS/DFS $O(V^2)$	O(grado) → BFS/DFS O(V + E)

Conviene la lista: la red es rala (18 de 66 tramos posibles, 27 %) y BFS, DFS y Dijkstra solo recorren vecinos. La matriz encaja naturalmente con Floyd, que de todos modos llena una matriz V×V. Con 12 localidades ambas son instantáneas; la diferencia aparecería con una red de miles de localidades con pocas rutas cada una.

BFS desde la base (Villa María)

Tramos	Localidades
1	Córdoba, Río Tercero, San Francisco, Río Cuarto
2	Villa Carlos Paz, Alta Gracia, Jesús María, Villa Dolores
3	Cosquín, Deán Funes
4 (fuera del pedido)	La Falda

BFS usa una cola propia sobre un arreglo de tamaño V (cada vértice entra una sola vez) y cuenta tramos, no km: por eso Río Cuarto está «a 1» (130 km) y Villa Carlos Paz «a 2» (182 km), mientras que San Francisco, también a 1 tramo, está a 150 km.

DFS: conexidad y corte de rutas

Un DFS recursivo desde cualquier vértice visita las 12 localidades: la red es conexa. El programa prueba cortar cada uno de los 18 tramos por separado y en ningún caso queda una localidad aislada: la red **no tiene puentes** (todo tramo está en algún ciclo). Para aislar una localidad hacen falta dos cortes; por ejemplo, Villa Dolores tiene solo dos tramos (Villa Carlos Paz y Río Cuarto) y, si se cortan ambos, el DFS desde la base deja de alcanzarla. San Francisco está en la misma situación (Córdoba y Villa María).

Dijkstra

Versión con arreglos, $O(V^2)$: en cada paso fija el vértice no fijado de menor distancia y relaja sus tramos. Con 12 vértices es más simple que con un heap e igual de rápido. Se guarda el previo de cada vértice para reconstruir el camino. Control con Córdoba: Río Cuarto a 212 km (Córdoba → Río Tercero → Río Cuarto), Villa Dolores a 176 km (por Villa Carlos Paz).

Destino	km	Camino desde Villa María
Río Tercero	70	Villa María → Río Tercero
Río Cuarto	130	Villa María → Río Cuarto
Córdoba	146	Villa María → Córdoba
San Francisco	150	Villa María → San Francisco
Alta Gracia	155	Villa María → Río Tercero → Alta Gracia
Villa Carlos Paz	182	Villa María → Córdoba → Villa Carlos Paz
Jesús María	197	Villa María → Córdoba → Jesús María
Cosquín	206	Villa María → Córdoba → Villa Carlos Paz → Cosquín
La Falda	226	… → Villa Carlos Paz → Cosquín → La Falda
Deán Funes	267	Villa María → Córdoba → Jesús María → Deán Funes
Villa Dolores	322	Villa María → Córdoba → Villa Carlos Paz → Villa Dolores

Ejemplo de por qué no alcanza con mirar pocos tramos: a Villa Dolores se llega en dos tramos pasando por Río Cuarto (130 + 210 = 340 km), pero el camino mínimo usa tres tramos, por Córdoba y Villa Carlos Paz, y suma 322 km. Y a Alta Gracia conviene ir por Río Tercero (155 km) y no por Córdoba (146 + 39 = 185 km), aunque Córdoba esté más cerca a un tramo.

Floyd

Tres ciclos anidados, $O(V^3) = 1.728$ pasos: después de la iteración k, `d[i][j]` es el mínimo usando como intermedios solo las localidades 0..k. Una matriz `siguiente[i][j]` guarda el primer paso para reconstruir

caminos. Coincide con Dijkstra en todas las filas comprobadas (Córdoba y Villa María).

```
       Cba  VCP   AG   JM   RT   VM   SF  Cos   LF   RC   DF   VD
Cba      0   36   39   51  102  146  205   60   80  212  121  176
VCP     36    0   35   87  120  182  241   24   44  230  139  140
AG      39   35    0   90   85  155  244   59   79  195  160  175
JM      51   87   90    0  153  197  256  111  131  263   70  227
RT     102  120   85  153    0   70  220  144  164  110  223  260
VM     146  182  155  197   70    0  150  206  226  130  267  322
SF     205  241  244  256  220  150    0  265  285  280  326  381
Cos     60   24   59  111  144  206  265    0   20  254  115  164
LF      80   44   79  131  164  226  285   20    0  274   95  184
RC     212  230  195  263  110  130  280  254  274    0  333  210
DF     121  139  160   70  223  267  326  115   95  333    0  279
VD     176  140  175  227  260  322  381  164  184  210  279    0
```

Plan de visitas

Con la red de 10.000 estaciones, las 12 localidades tienen estaciones críticas. Vecino más cercano sobre la tabla de Floyd, saliendo y volviendo a Villa María:

```
Villa María → Río Tercero (70) → Alta Gracia (85) → Villa Carlos Paz (35) → Cosquín (24) → La
Falda (20)
→ Córdoba (80) → Jesús María (51) → Deán Funes (70) → Villa Dolores (279) → Río Cuarto (210)
→ San Francisco (280) → Villa María (150)                                    Total: 1.354 km
```

Para medir qué tan lejos queda, el programa calcula también el recorrido óptimo con programación dinámica sobre subconjuntos (Held-Karp, $O(2^k \cdot k^2)$, viable solo porque k = 11): **1.215 km** (Villa María → San Francisco → Córdoba → Jesús María → Deán Funes → La Falda → Cosquín → Alta Gracia → Villa Carlos Paz → Villa Dolores → Río Cuarto → Río Tercero → Villa María). **El vecino más cercano hace 139 km más (11,4 %).**

Por qué no garantiza el óptimo: es un algoritmo goloso. En cada paso elige lo más cercano desde donde está sin considerar lo que falta. Sale hacia el oeste por los tramos cortos (Río Tercero, Alta Gracia, Villa Carlos Paz, Cosquín) y deja para el final las localidades de los extremos: desde Deán Funes tiene que cruzar a Villa Dolores (279 km), bajar a Río Cuarto y atravesar la provincia hasta San Francisco (280 km). Los cuatro últimos tramos suman 919 km. El óptimo empieza por San Francisco, que está a 150 km de la base, y recorre el anillo sin volver sobre sus pasos. Encontrar el óptimo es el problema del viajante, que no tiene algoritmo eficiente conocido: con 30 localidades Held-Karp necesitaría unos $2^{30} \cdot 30^2 \approx 10^{12}$ pasos. Con menos localidades la diferencia baja pero no desaparece: con 30 estaciones hay 5 localidades críticas y el vecino más cercano hace 1.131 km contra 1.077 del óptimo (5 %).

5. M4 · Transmisión comprimida (Huffman)

Diseño

Sigue la estructura de la lectura: clase `Nodo` (símbolo, frecuencia, hijos; comparable por frecuencia), `construirArbolHuffman` (una hoja por símbolo en una `PriorityQueue`; se sacan los dos de menor frecuencia y se unen bajo un padre con la suma, hasta que queda uno), `codificar` (recorre el árbol: 0 a la izquierda, 1 a la derecha; la tabla queda en un `HashMap`) y `decodificar` (baja bit a bit desde la raíz y, al llegar a una hoja, emite el símbolo y vuelve a la raíz). Para frecuencias iguales se desempata por orden de creación, así el árbol es siempre el mismo. Con un solo símbolo distinto se le agrega un padre para que su código sea «0».

Además del cálculo en bits, `ArchivoHuffman` escribe el archivo comprimido real (.huf): tabla de frecuencias, cantidad de bits y los bits empaquetados de a 8 por byte. El receptor reconstruye el mismo árbol con las frecuencias.

Caso de verificación

`WWXWYZWX`: W = 4, X = 2, Y = 1, Z = 1 → códigos W = 0, X = 10, Y = 110, Z = 111. Codificado: `00100110111010`, **14 bits** contra 64 con 8 bits fijos. Decodificado idéntico. ✓

Archivo de mediciones (5.000 líneas)

Medida	Valor
Caracteres / tamaño original (8 bits por carácter)	254.642 / 2.037.136 bits
Tamaño comprimido (bits de datos)	1.071.484 bits
Bits promedio por carácter	4,21
Tasa de compresión	52,6 % del original (ahorro 47,4 %)
Archivo .huf real, con la tabla de frecuencias	134.080 bytes (52,7 %)
Símbolos distintos / códigos libres de prefijo	22 / sí
Descomprimido desde el .huf idéntico al original	sí

Los más frecuentes reciben 3 bits (`;` 30.000 veces, `0` 28.112, `1` 25.906) y los menos frecuentes 6 (`\n`, `:`, `C`, `H`, 5.000 veces cada uno).

¿Por qué se comprime bastante pero no tanto como un texto con muchos repetidos?

Huffman gana cuando las frecuencias son muy desiguales: los símbolos muy frecuentes reciben códigos muy cortos. El archivo de mediciones tiene 22 símbolos con frecuencias bastante parejas: los 10 dígitos aparecen entre 5.600 y 28.000 veces, y la estructura fija de cada línea (`CBA-`, `;T=`, `;H=`, …) reparte los separadores en cantidades similares. Con 22 símbolos casi equiprobables el mínimo sería log2 22 ≈ 4,46 bits por carácter; Huffman logra 4,21 porque hay algo de desigualdad (el `;` y los dígitos 0, 1 y 2 de las fechas dominan), pero no mucho más. En `WWXWYZWX`, en cambio, la mitad de los caracteres son W y se codifican con 1 bit: 1,75 bits por carácter (78 % de ahorro). Además Huffman codifica cada carácter por separado: no aprovecha que `CBA-` o la fecha se repiten idénticas en todas las líneas, algo que sí explotan los compresores por diccionario.

6. M5 · Guardias rotativas (Josephus)

- **Simple, O(N·M):** `LinkedList` con los técnicos 1..N y un `Iterator`. Para cada eliminación se avanza M + 1 posiciones (si se termina la lista, se pide un iterador nuevo: la ronda es circular) y se llama a `remove()`, que es O(1) porque el iterador ya está sobre el nodo.
- **Eficiente, O(N log N):** no camina M pasos: calcula la posición del próximo eliminado, `rango = (rango + M) mod quedan` (0 equivale a «quedan»), y la busca en un ABB donde cada nodo guarda el tamaño de su subárbol. Así el k-ésimo se encuentra y se elimina en O(altura). El árbol se arma balanceado con 1..N (altura ≈ log2 N) y las eliminaciones solo pueden achicar la altura.

Caso de verificación: N = 5, M = 1 → salen 2, 4, 1, 5 y queda el 3, con las dos implementaciones. ✓

Calendario (N = 12, M = 2)

```
Ronda: 1.Ana 2.Bruno 3.Carla 4.Diego 5.Elena 6.Facundo 7.Gabriela 8.Hernán 9.Inés 10.Julián
11.Karina 12.Lucas
Fin de semana 1 → Carla · 2 → Facundo · 3 → Inés · 4 → Lucas · 5 → Diego · 6 → Hernán · 7 → Ana
· 8 → Gabriela · 9 → Bruno · 10 → Karina · 11 → Elena · Suplente de emergencias: Julián
```

Es justo porque la regla es fija y pública, no depende de nadie, y verificable porque cualquiera puede repetir la ronda a mano y llegar al mismo calendario.

Experimento obligatorio

N	M	Lista O(N·M)	Árbol O(N log N)	Lista / árbol
1.000	2	0,25 ms	0,09 ms	2,7
10.000	2	0,58 ms	1,00 ms	0,6
100.000	2	2,7 ms	5,5 ms	0,5
1.000	100	0,36 ms	0,05 ms	7,0
10.000	100	2,0 ms	0,63 ms	3,2
100.000	100	30,7 ms	10,2 ms	3,0
1.000	1.000	3,7 ms	0,04 ms	93,9
10.000	1.000	20,2 ms	0,66 ms	30,4
100.000	1.000	281,6 ms	10,0 ms	28,1

Cuándo importa la diferencia. El costo de la lista es proporcional a N·M; el del árbol, a N·log N y no depende de M. Con M chico (nuestro M = 2) la lista hace muy poco trabajo por eliminación y, salvo con N chico, gana, porque cada paso del árbol es más caro (recursión, actualizar tamaños). A medida que M crece la lista se vuelve proporcionalmente más lenta y el árbol no cambia: con M = 1.000 el árbol es entre 28 y 94 veces más rápido. Con N = 100.000 y M = 1.000 la lista recorre unos 10^8 nodos (283 ms); con M = N sería del orden de 10^{10} y ya no sería práctico. Para 12 técnicos cualquiera de las dos es instantánea.

7. M6 · Simulación y pruebas

- **Generación reproducible:** todo sale de `new Random(semilla del grupo)`. Los números de estación se obtienen mezclando 0..9999 con Fisher-Yates parcial (sin repetidos, O(n)); localidad, tipo, batería (0..100) y días sin visita (0..180), con `nextInt`. Correr dos veces el programa da exactamente los mismos resultados.
- **secuenciasIguales / secuenciasDistintas:** dos generadores con semilla 2026 producen los mismos 1.000 valores (97, 96, 26, 54, 36, …); con 2027 la secuencia es otra (27, 63, 22, 23, 3, …).
- **Uniformidad:** 1.000.000 de baterías. Esperado por valor: 9.901. Por intervalos de 10 el desvío máximo es 0,50 %; por valor, 2,35 %. Chi-cuadrado = 84,15 con 100 grados de libertad; el crítico al 5 % es 124,34, así que no hay evidencia contra la uniformidad.
- **Tiempos:** `System.nanoTime`. Antes de medir se hace una pasada de calentamiento (la JVM compila el código caliente sobre la marcha) y se toma el mejor de 3 a 5 repeticiones, que es el menos afectado por el recolector de basura y otros procesos. El informe se guarda en `resultados/informe_rendimiento_grupoN.txt`.

8. Complejidad de las operaciones principales

Estructura / algoritmo	Operación	Promedio	Peor caso
Tabla hash (abierta)	insertar, buscar, eliminar	O(1) con λ acotado	O(n)
Tabla hash	rehashing	O(n), O(1) amortizado por inserción	O(n)
ABB	insertar, buscar, eliminar	O(log n)	O(n) (claves ordenadas)
AVL	insertar, buscar, eliminar	O(log n)	O(log n)
ABB / AVL	recorridos, altura, nodos, hojas, críticas	O(n)	O(n)
ABB / AVL	rango [A, B] con k resultados	O(log n + k)	O(n) en ABB
BFS, DFS	lista / matriz	$O(V + E)$ / $O(V^2)$	
Dijkstra (arreglos)	desde un origen	$O(V^2)$	
Floyd	todos los pares	$O(V^3)$	
Vecino más cercano / Held-Karp	k localidades	$O(k^2)$ / $O(2^k \cdot k^2)$	
Huffman	construir árbol (σ símbolos) / codificar texto de n caracteres	O(σ log σ) / O(n)	
Josephus	lista + iterador / árbol con rangos	O(N·M) / O(N log N)	

9. Casos de verificación

Caso	Esperado	Obtenido
Huffman WWXWYZWX	64 bits → 14 bits	14 bits, decodificado idéntico ✓
Josephus N = 5, M = 1	salen 2, 4, 1, 5; queda 3	[2, 4, 1, 5, 3] con ambas implementaciones ✓
Dijkstra desde Córdoba (control)	caminos mínimos de la tabla	coincide con Floyd y con el cálculo a mano ✓
Matriz contra lista	mismos resultados	Dijkstra idéntico con ambas ✓
Inorden del AVL con 10.000 claves	estrictamente creciente	sí ✓
Baja y alta en tabla hash + AVL	la baja deja de encontrarse; el alta vuelve a aparecer	sí, reutilizando la celda borrada ✓
Archivo .huf	descomprimido = original	sí, byte a byte ✓
Semillas	misma semilla, misma secuencia	sí ✓

10. Conclusiones

- **No hay una estructura mejor en general, sino una mejor para cada pregunta.** La tabla hash responde «¿dónde está este código?» en 1 a 2 sondeos, y el AVL responde «¿cuáles hay entre 340 y 350?» y «¿cuáles están críticas, en orden?». RedSens mantiene las dos.
- **Lo que dice la teoría se ve en los números**, siempre que la implementación respete los supuestos: el agrupamiento primario aparece con exploración lineal y λ alto, el ABB degenera con datos ordenados

exactamente como el censo los entrega, y Josephus con lista crece con M mientras que con árbol no.

- **Medir sirve para encontrar errores de diseño.** La primera función hash parecía razonable y pasaba todas las pruebas funcionales, pero los sondeos medidos no coincidían con la fórmula. Esa diferencia llevó a encontrar que Horner solo deja juntos los códigos correlativos.
- **Una heurística rápida no es una solución óptima.** El vecino más cercano arma el plan en microsegundos, pero en este caso recorre 139 km (11 %) más que el óptimo. Para 12 localidades se puede calcular el óptimo; para la red completa de la provincia habría que aceptar una heurística y mejorarla.
- **Huffman depende de la distribución de los símbolos.** Con frecuencias parejas el ahorro ronda el 47 %; con un símbolo dominante, como en WWXWYZWX, llega al 78 %.